

Rapport de Progression : Conversion d'un document manuscrit en une version texte

CASANOVA KINKELA Lise
N°SCEI 22685

Positionnement thématique:

Sciences industrielles (Traitement du Signal)

Informatique (Informatique pratique)

Sommaire

Projet.....	3
Déroulement.....	4
Traitement de l'image.....	5
Convertir une image en jpg et bmp.....	5
Convertir une image en nuance de gris.....	6
Réduire le traitement d'une image.....	11
Filtre de convolution.....	11
Seuillage dynamique.....	14
Modélisation des séquences.....	18
Reconnaissance.....	18
Encodeur MDLSTM.....	19
Encodeur.....	19
Décodeur.....	21
Attention.....	21
(Reconnaissance final).....	22
Source.....	23

Projet

J'ai été confrontée durant ma scolarité (étant dyslexique, dysphasique et dysorthographique) à des difficultés lors des prises de notes qui auraient pu être atténuées par des dispositifs d'accompagnement. Je me suis intéressée à ces techniques qui permettent de se concentrer pleinement sur les cours:

Convertir de l'écriture manuscrite en numérique.

L'objectif de ce travail est de réaliser une imitation simplifiée d'un modèle de reconnaissance de caractères en s'inspirant fortement de l'idée et du but des architectures de réseaux neuronaux. Pour cela, je reprends le concept des **KNN moyens**, une méthode d'intelligence artificielle basée sur l'approximation des voisins les plus proches, ainsi que des techniques de **traitement d'image** permettant d'extraire et de prétraiter les caractéristiques visuelles des caractères manuscrits. Enfin, une évaluation de la qualité de la reconnaissance sera réalisée à l'aide d'un **code de similarité**, permettant de comparer les résultats obtenus avec des références connues.

Déroulement

Je me suis intéressée aux travaux de Melchior OBAME OBAME et Saad BENDAOU, notamment à leur projet de deuxième année (2018-2019) portant sur l'utilisation des réseaux de neurones pour la reconnaissance de l'écriture manuscrite [\[1\]](#). L'analyse de cette documentation m'a aidée à structurer ma démarche en m'inspirant de différentes architectures de réseaux neuronaux :

- **CNN (Convolutional Neural Network)**: Extraction des caractères (traitement de l'image)
- **LSTM (Long Short-Term memory)**: Modélisation des séquences
- **CTC (Connectionist Temporal Classification)**: Reconnaissance final

Traitement de l'image

Le traitement d'image permet la validation de l'image avant d'utiliser l'IA.

Convertir une image en jpg et bmp

Recherche sur les format des images [\[3\]](#):

1. Formats Bitmap: stockage en pixels ce qui est adaptés aux photos ou les images complexes
 - a. Format sans perte (Lossless):
 - i. **BMP (Bitmap): utilisé pour le stockage ou le web**
 - Pas de compression
 - Accès direct aux pixels
 - ii. PNG (Portable Network Graphics): utilisé pour les logos ou icônes
 - Compression sans perte
 - Supporte la transparence (canal alpha)
 - iii. TIFF (Tagged Image File Format): utilisé pour les impressions et photographies
 - Fichier très lourds
 - iv. GIF (Graphics Interchange Format): utilisé pour les images animées
 - Limité à 256 couleurs sur 8 bits
 - Supporte la transparence et les animations
 - b. Formats avec compression destructive (Lossy):
 - i. **JPEG/ JPG (Joint Photographic Experts Group): utilisé pour les photos ou images en haute résolution**
 - Compression avec perte ajustable
 - ii. WebP (Google Web Picture Format): utilisé pour le web, remplace le jpeg, png, et gif)
 - Supporte la compression avec et sans perte
 - Supporte la transparence et les animations
 - iii. HEIF/ HEIC (High Efficiency Image Format): utilisé par Apple pour les photos Iphone
 - Compression plus efficace que JPEG
 - Compatibilité moins universelle que JPEG
2. Formats Vectoriels: stockage en formes mathématiques ce qui est évolutifs sans perte de qualité
 - i. SVG (Scalable Vector Graphics): utilisé pour les images vectorielle sur le web
 - Basé sur XML
 - Redimensionnable sans perte de qualité
 - ii. EPS (Encapsulated PostScript): utilisé en impression et publication assistée par ordinateur
 - Supporte le texte et des graphiques vectoriels
 - iii. PDF (Portable Document Format): utilisé pour l'impression et la distribution de documents
 - Supporte les images bitmap et vectorielle
2. Formats Spécifiques:
 - i. RAW (Format brut des appareils photo): utilisé en photographie professionnelle

- Contient les données brutes du capteur
- ii. ICO (Icon Format): utilisé pour les icônes sous Windows
 - Contient plusieurs tailles et résolutions d'icônes dans un seul fichier
- iii. PSD (Photoshop Document): utilisé par Adobe Photoshop
 - Contient des calques, des masques et des effets
- iv. XCF (eXperimental Computing Facility): utilisé par GIMP
 - Similaire à PSD

L'image que j'utilise vient d'un appareil photo (téléphone) ainsi il sera en format **JPEG/ JPG**. Cependant le format le plus adapté au traitement d'image est le **BMP**.

Fonction: [Convert](#)

Convertir une image en nuance de gris

Théorie

Utilisation d'un TP de PTSI.

Il existe plusieurs méthodes, dont 3 principaux pour trouver les coefficient de pondération:

Moyenne simple

Cette méthode fait perdre le contraste et ne compte pas la perception

tel que les coefficients de pondération sont $[\frac{R+G+B}{3}, \frac{R+G+B}{3}, \frac{R+G+B}{3}]$, avec RGB la valeur de la pixel à la ligne 51.

Fonction: [simple average](#)

Luminosité maximale

Cette méthode garde la composante la plus forte, ainsi on perd les détails

tel que les coefficients de pondération sont $[Max(R, G, B), Max(R, G, B), Max(R, G, B)]$, avec RGB la valeur de la pixel à la ligne 51.

Fonction: [Maximum brightness](#)

Luma (NTSC)

J'utilise la méthodologie de **Luma (NTSC)**, qui prend en compte la sensibilité de l'œil humain par rapport à la couleur. Cette méthode est utilisée principalement par les systèmes de télévision et de traitement d'image.

La méthode **Luma (NTSC)** compense les différentes perception de luminosité en fonction des couleurs en pondérant chaque couleur selon sa contribution à la luminosité perçue.

Tout d'abord elle se base sur l'espace **CIE XYZ** [4], elle a été définie en 1931 par la Commission Internationale de l'Eclairage (CIE). A la différence du RGB, la CIE XYZ se base sur la perception humaine des couleurs, ce qui en fait un modèle fondamental pour la colorimétrie.

- X : Composante représentant un mélange des sensibilités des cônes (proche du rouge mais pas identique à R).
- Y : **Correspond exactement à la luminance perçue (c'est la plus importante pour la vision en noir et blanc).**
- Z : Représente principalement la sensibilité au bleu (correspond au bleu profond).

Ainsi le Y est la composante qui m'intéresse, elle est définie comme la **luminance** perçue par l'œil humain.

Il existe une matrice de passage M, tel que:

$$\begin{bmatrix} X \\ Y \\ Z \end{bmatrix} = [M] \begin{bmatrix} R \\ G \\ B \end{bmatrix}$$

Soit $(x_r, y_r, x_g, y_g, x_b, y_b, X_w, Y_w, Z_w) \in \mathbb{R}^9$:

Avec (x_r, y_r) la coordonnée chromatique rouge (red) du pixel

(x_g, y_g) la coordonnée chromatique vert (green) du pixel

(x_b, y_b) la coordonnée chromatique bleu (blue) du pixel

(X_w, Y_w, Z_w) Valeur de référence correspondant au blanc (white)

$$[M] = \begin{bmatrix} S_r X_r & S_g X_g & S_b X_b \\ S_r Y_r & S_g Y_g & S_b Y_b \\ S_r Z_r & S_g Z_g & S_b Z_b \end{bmatrix}$$

Pour trouver les équations de X_i, Y_i et Z_i avec $i \in [r, g, b]$:

je passe par la conversion xyY à XYZ:

Sachant que la chromaticité est définie comme les proportions des composantes X, Y, Z dans le mélange total, j'obtiens :

$$\begin{aligned} x &= \frac{X}{X+Y+Z} \\ y &= \frac{Y}{X+Y+Z} \\ z &= \frac{Z}{X+Y+Z} \end{aligned}$$

De plus que $x + y + z = 1$, z peut être déterminé en fonction de x et y, donc il suffit de x et y pour caractériser la chromaticité.

J'ai les équations mais je l'ai veu en fonction de x et y :

$$\begin{aligned}
x &= \frac{X}{X+Y+Z} \Leftrightarrow x(X+Y+Z) = X \quad L_1 \rightarrow \frac{L_1}{L_2} \\
y &= \frac{Y}{X+Y+Z} \quad y(X+Y+Z) = Y \\
&\Leftrightarrow \frac{x}{y} = \frac{X}{Y} \\
&\quad y(X+Y+Z) = Y \\
&\quad X = \frac{x}{y} Y \\
&\Leftrightarrow y\left(\frac{x}{y} Y + Y + Z\right) = Y \\
&\quad X = \frac{x}{y} Y \\
&\quad xY + yY + yZ - Y = 0 \\
&\Leftrightarrow X = \frac{x}{y} Y \\
&\quad Y(x+y-1) + yZ = 0 \\
&\quad X = \frac{x}{y} Y \\
&\Leftrightarrow Z = Y \frac{(1-x-y)}{y}
\end{aligned}$$

Et pour la conversion de RGB à XYZ, je pose $Y = 1$ pour normaliser les couleurs, pour être comparable indépendamment de leur intensité absolue, pour dépendre seulement de la chromaticité. J'obtiens donc:

Soit $i \in [r, g, b]$, tel que:

$$\begin{aligned}
X_i &= \frac{x_i}{y_i} \\
Y_i &= 1 \\
Z_i &= \frac{1-x_i-y_i}{y_i}
\end{aligned}$$

Attention, comme Y est normalisé les autres valeurs doivent se trouver entre $[0;1]$.

$$\begin{bmatrix} S_r \\ S_g \\ S_b \end{bmatrix} = \begin{bmatrix} X_r & X_g & X_b \\ Y_r & Y_g & Y_b \\ Z_r & Z_g & Z_b \end{bmatrix}^{-1} \begin{bmatrix} X_w \\ Y_w \\ Z_w \end{bmatrix}$$

Pour trouver les équations de S_r , S_g , et S_b :

Donnée:

Pour X_w , Y_w et Z_w :

Nom de la référence blanc	X	Y	Z
A	1.09850	1.00000	0.35585
B	0.99072	1.00000	0.85223
C	0.98074	1.00000	1.18232
D50	0.96422	1.00000	0.82521
D55	0.95682	1.00000	0.92149
D65	0.95047	1.00000	1.08883
D75	0.94972	1.00000	1.22638
E	1.00000	1.00000	1.00000
F2	0.99186	1.00000	0.67393
F7	0.95041	1.00000	1.08747
F11	1.00962	1.00000	0.64350

Pour x_r, y_r, x_g, y_g, x_b et y_b : Je prends comme valeur de référence du blanc **E**.

	x	y
Rouge (Red)	0.7350	0.2650
Vert (Green)	0.2740	0.7170
Bleu (Blue)	0.1670	0.0090

J'obtiens comme valeurs :

	X	Y	Z
Rouge (Red)	$X_r = \frac{0.7350}{0.2650} = 2.774$	$Y_r = 1$	$Z_r = \frac{1-0.7350-0.2650}{0.2650} = 0$
Vert (Green)	$X_g = \frac{0.2740}{0.7170} = 0.3821$	$Y_g = 1$	$Z_g = \frac{1-0.2740-0.7170}{0.7170} = 0.01255$
Bleu (Blue)	$X_b = \frac{0.1670}{0.0090} = 18.56$	$Y_b = 1$	$Z_b = \frac{1-0.1670-0.0090}{0.0090} = 91.56$

Les matrices :

$$\begin{bmatrix} X_r & X_g & X_b \\ Y_r & Y_g & Y_b \\ Z_r & Z_g & Z_b \end{bmatrix} = \begin{bmatrix} 2,774 & 0,3821 & 18,56 \\ 1 & 1 & 1 \\ 0 & 0,01255 & 91,56 \end{bmatrix}$$

$$\begin{bmatrix} X_r & X_g & X_b \\ Y_r & Y_g & Y_b \\ Z_r & Z_g & Z_b \end{bmatrix}^{-1} = \begin{bmatrix} 0,418 & -0,159 & -0,083 \\ -0,418 & 1,159 & 0,072 \\ 0 & 0 & 0,011 \end{bmatrix}$$

J'obtiens les coefficients:

$$\begin{bmatrix} S_r \\ S_g \\ S_b \end{bmatrix} = \begin{bmatrix} 0,418 & -0,159 & -0,083 \\ -0,418 & 1,159 & 0,072 \\ 0 & 0 & 0,011 \end{bmatrix} \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

J'obtiens la matrice M:

$$[M] = \begin{bmatrix} 0,176 \times 2,774 & 0,813 \times 0,3821 & 0,011 \times 18,56 \\ 0,176 \times 1 & 0,813 \times 1 & 0,011 \times 1 \\ 0,176 \times 0 & 0,813 \times 0,01255 & 0,011 \times 91,56 \end{bmatrix}$$

$$\begin{bmatrix} S_r \\ S_g \\ S_b \end{bmatrix} = \begin{bmatrix} 0,176 \\ 0,813 \\ 0,011 \end{bmatrix}$$

$$[M] = \begin{bmatrix} 0,488 & 0,311 & 0,204 \\ 0,176 & 0,813 & 0,011 \\ 0 & 0,010 & 1,007 \end{bmatrix}$$

J'obtiens à 10^{-3} près une matrice de conversion entre RGB à XYZ, cependant seulement la ligne de Y m'intéresse ainsi, j'obtiens pour mon code les coefficients **[0.176, 0.813, 0.011]** de la ligne 51.

Fonction: [Luma](#)

Expérience

Je vais effectuer 4 types d'expérience pour cette partie:

- Comparaison visuelle
- Comparaison des histogramme
- Comparaison de l'entropie
- Comparaison du temps de calcul

Fonction: [Comparison of shades of grey](#)

Réduire le traitement d'une image

Annexe: [Séparation d'une image en plusieurs couches](#)

Convertir une image d'entier en binaire

Fonction: [Convert integer to binary](#)

Convertir une image de binaire en entier

Fonction: [Convert binary to integer](#)

Réduction du nombre de couche

Fonction: [Bit plane slicing](#)

Maintenant je peux comparer la visibilité de l'image en fonction du nombre de couches gardées.

Fonction: [Comparison coat number](#)

Mais les résultats ne sont pas concluants. Ainsi je n'utiliserai pas de réduction de couches par la suite.

Filtre de convolution

Les filtres de convolution sont liés au seuillage dynamique. Il est souvent associé dans le calcul final pour grossir le trait.

Filtre gaussien

Le filtre gaussien se place avant le filtre de Sobel (ou de Canny), il permet de réduire les faux positifs dus à du bruit.

La formule du noyau de la gaussienne:

$$w(i, j) = K e^{\frac{-i^2 + j^2}{2\sigma^2}}$$

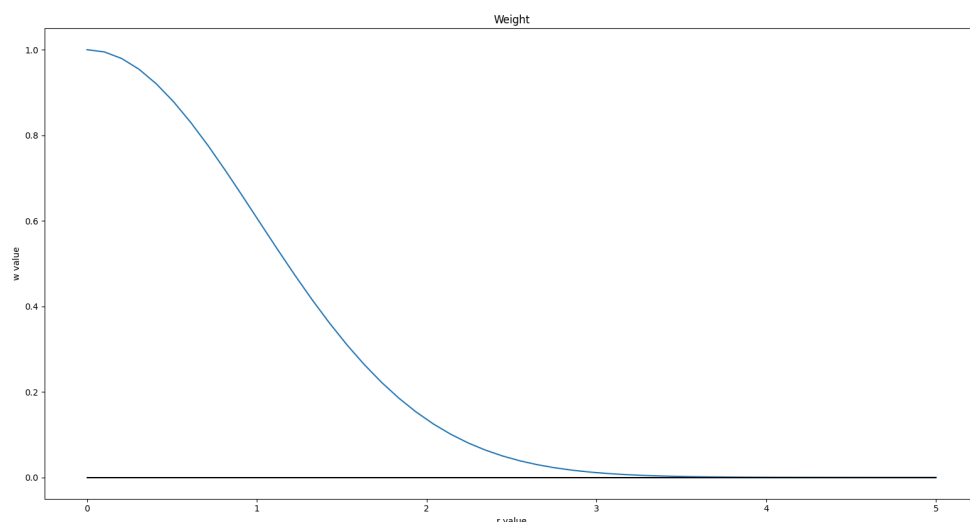
avec K une constante

L'avantage du filtre gaussien est qu'il peut être séparable ce qui permet de réduire les calculs donc réduire le temps de calcul du programme.

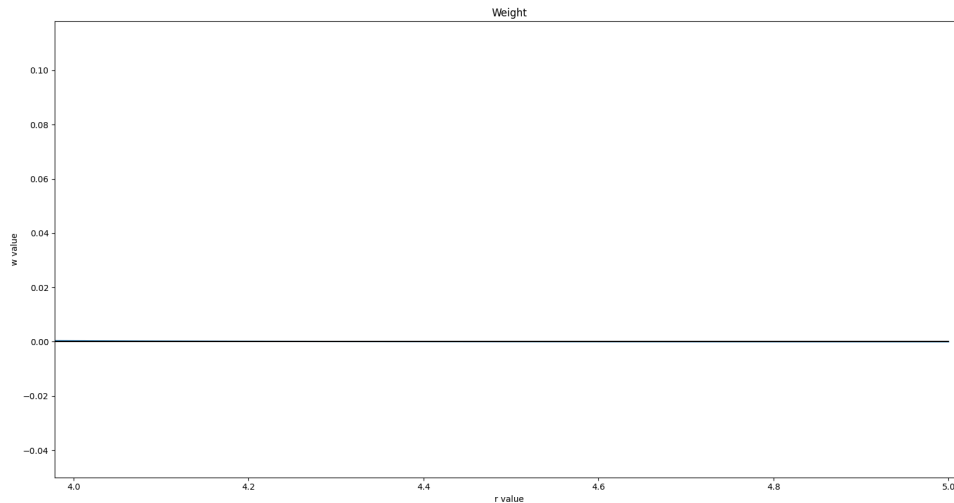
On pose $r = [i^2 + j^2]^{1/2}$, tel que:

$$w(i, j) = K e^{\frac{-r^2}{2\sigma^2}}$$

Je peux déduire en regardant la formule que plus r est grand plus w est petit donc si je prend un nombre important de voisin (pour le graphique K = 1 et $\sigma = 1$):



Zoom:



Je peux voir qu'au 4ème voisin le poids est négligeable.

Fonction: [Weight of the Gaussian formula](#)

Fonction: [Gaussian Filter Kernels](#)

Filtre Laplacien

Le but de filtre est d'améliorer la netteté de l'image, pour cela on s'intéresse à l'isotropie du noyaux (qui est indépendant de la direction donc filtre séparable).

Pour comprendre le filtre je me place en 2D avec la formule du Laplacien:

$$\nabla^2 f = \frac{\partial^2 f}{\partial x^2} + \frac{\partial^2 f}{\partial y^2}$$

Comme étudié dans la partie [les dérivées](#) qui me dit:

$$\frac{\partial^2 f}{\partial x^2} = f(x + 1, y) + f(x - 1, y) - 2f(x, y)$$

Comme dans la direction y, il n'y a aucune différence:

$$\frac{\partial^2 f}{\partial y^2} = f(x, y + 1) + f(x, y - 1) - 2f(x, y)$$

Il suffit de remplacer dans la formule du Laplacien:

$$\nabla^2 f = f(x + 1, y) + f(x - 1, y) + f(x, y + 1) + f(x, y - 1) - 4f(x, y)$$

0	1	0
1	-4	1
0	1	0

Ainsi si je prends les coefficient en les considérant comme le poids, j'obtiens:

Je refais les même calculant en tournant l'image à 45° puis je la somme à celle d'avant et j'obtiens:

1	1	1
1	-8	1
1	1	1

Attention il ne faut pas remplacer l'image obtenue par le Laplacien, il faut l'ajouter à l'image si je veux une bonne utilisation du filtre.

$$g(x, y) = f(x, y) + c \left[\nabla^2 f(x, y) \right]$$

Le Laplacien peut être aussi utilisé en 1D permettant de réduire le temps de calcul.

Seuillage dynamique

Le seuillage dynamique rend l'image en binaire (soit un pixel blanc ou un pixel noir).

Il existe plusieurs méthodes [\[5\]](#):

- Seuillage manuel
- Seuillage locale:
 - Seuillage moyenne locale
 - Seuillage médiane locale
 - Seuillage pondérée gaussienne
- Seuillage adaptatif:
 - Seuillage de Niblack
 - Seuillage de Sauvola
 - Seuillage de Wolf et Jolion

Théorie

Seuillage Manuel

C'est une méthode nommée aussi seuillage global qui est utilisée pour binariser une image avec une valeur seuil global.

La méthode du seuillage global consiste à poser un seul seuil nommé T pour tous les pixels de l'image. Cependant il n'y a aucune information sur le choix de T (recherche empirique).

Fonction: [Manual](#)

Seuillage local

La méthode du seuillage local consiste à poser un seuil à chaque région de l'image.

Pour cela il existe plusieurs méthodes pour calculer le seuil, la plupart du temps nous utilisons la méthode utilisée avant.

Tout au long, je parlerais d'une fenêtre (carré de préférence) contenant les pixels voisins (sans exclure le pixel centrale) avec n le nombre de voisins en sachant que nous perdons des pixels plus précisément $n(l+c-4n)$ (l est le nombre de ligne et c est le nombre de colonne) donc plus n est grand plus l'image se réduit de plus x est une valeur sur les lignes de l'image et y est une valeur sur les colonnes de l'image. Quant à N le nombre de valeur de la fenêtre et $I(x,y)$ correspondant à l'intensité du pixel.

Moyenne Locale

La moyenne locale est obtenue en prenant une fenêtre et en faisant la moyenne d'intensité de toutes les pixels de la fenêtre. Si le pixel central est plus petit que la moyenne des pixels de la fenêtre alors on change ce pixel en [0, 0, 0] (correspondant à la couleur blanche) sinon en [255, 255, 255] (correspondant à la couleur noire). Je peux rajouter une constante d'ajustement, pour pouvoir être plus précise.

$$M(x, y) = \frac{1}{N} \sum_{(i, j) \in [x-n : x+n] \times [y-n : y+n]} I(i, j)$$

Fonction: [Local average](#)

Médiane Locale

La médiane locale est obtenue en prenant les valeurs de la fenêtre, puis j'organise les pixels par ordre de croissance pour en extraire la médiane. Si le pixel central est plus petit que la médiane des pixels de la fenêtre alors on change ce pixel en [0, 0, 0] (correspondant à la couleur blanche) sinon en [255, 255, 255] (correspondant à la couleur noire).

Fonction: [Local median](#)

Seuillage pondérée gaussienne

Le seuillage pondérée gaussienne [2] ne donne pas le même poids à chaque pixel, c'est-à-dire que j'applique une pondération gaussienne aux pixels voisins (dans une fenêtre), ajoutant de l'importance sur les pixels les plus proches. Le but est de lisser l'image en réduisant le bruit.

Pour cela il faut effectuer un calcul de dispersion des intensités des pixels dans une fenêtre:

$$\sigma^2(x, y) = \frac{1}{N} \sum_{(i, j) \in [x-n: x+n] \times [y-n: y+n]} (I(i, j) - M(x, y))^2$$

Puis j'obtiens la formule qui permet de déterminer le seuil avec le filtre gaussien (G):

$$M(x, y) = \frac{1}{N} \sum_{(i, j) \in [x-n: x+n] \times [y-n: y+n]} G(i, j) \times I(i, j)$$

Si le pixel central est plus petit que le résultat alors on change ce pixel en [0, 0, 0] (correspondant à la couleur blanche) sinon en [255, 255, 255] (correspondant à la couleur noire).

Fonction: [Weighted Gaussian threshold](#)

Seuillage Adaptatif

Cette méthode se rapproche de celle du seuillage local, la différence se trouve sur la découverte.

Seuillage de Niblack

Soit k est le coefficient de pondération et M la moyenne trouvé dans le seuillage pondérée gaussienne [6]:

$$T(x, y) = M(x, y) + k \times \sigma(x, y)$$

Le coefficient de pondération k est choisie expérimentalement et elle sera négative.

Fonction: [Niblack](#)

Seuillage de Sauvola

Cette méthode est une amélioration de celle de Niblack [6], avec R la valeur de normalisation de l'écart-type:

$$T(x, y) = M(x, y) \times \left(1 + k \times \left(\frac{\sigma(x, y)}{R - 1} - 1 \right) \right)$$

La valeur de normalisation de l'écart-type est la moitié de l'intensité maximale en 8 bits, par défaut $R = 128$.

Fonction: [Sauvola](#)

Seuillage de Wolf et Jolion

Cette méthode est une amélioration de celle de Sauvola, avec $\text{Max}(I)$ et $\text{Min}(I)$ correspondant respectivement à l'intensité maximum et l'intensité minimal de l'image:

$$T(x, y) = M(x, y) \times (1 - k) + k \times \left(\frac{\sigma(x, y)}{R} \times (\text{Max}(I) - I(x, y)) \right)$$

Je garde les mêmes valeurs que précédemment pour k et R .

Fonction: [Wolf](#)

Conclusion

Les seuillages les plus intéressants sont les seuillages dynamiques, mais le choix entre les 3 se fait visuellement. De même pour le choix des coefficients. Dans chaque cas la valeur de k se trouvera entre $[0,2; 0,5]$ avec le 0,2 qui est plus sélectif sur les trait et le 0,5 moins sélectif. Pour n correspondant au nombre de voisin, plus le nombre est grand plus mon écriture ressort (donc il n'y a plus de trait parasite sur mon image), cependant si la valeur est trop élevée non seulement je perdrais mon trait d'écriture mais en plus le temps de calcul sera grand. La complexité des trois fonction est $O(H \times L \times n^2)$ avec H la hauteur, L la largeur et n le nombre de voisin. Mais pour l'instant sur le test fait le sauvola est celui qui rend le mieux.

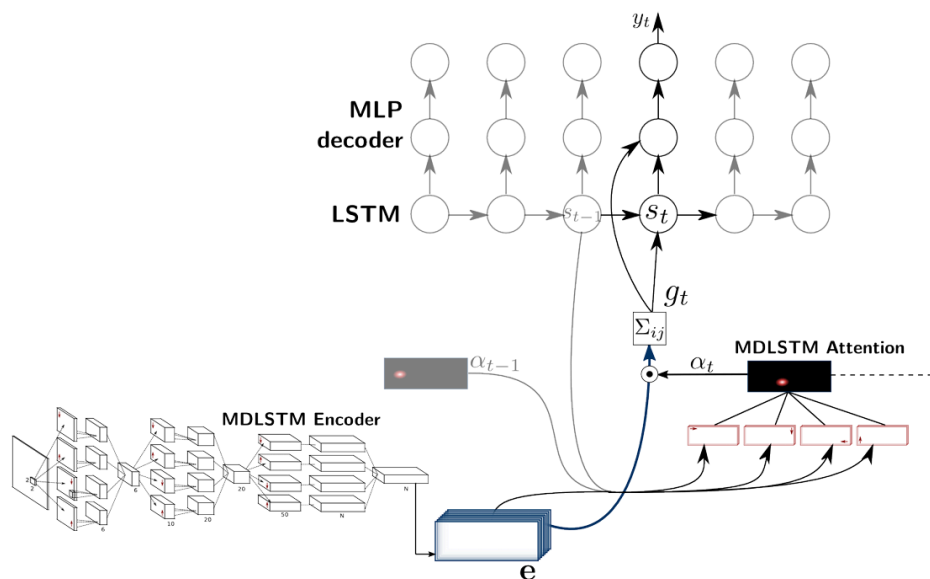
Modélisation des séquences

La construction de cette partie est basée sur l'étude de Théodore Bluche, Jérôme Louradour et Ronaldo Messina [7].

Reconnaissance

J'utilise le MDLSTM - RNN (Multi-Dimensional Long Short-Term Memory Recurrent Neural Networks) avec un CTC (Connectionist Temporal Classification). Le MDLSTM - RNN est une alternative au LSTM (Long Short Term Memory).

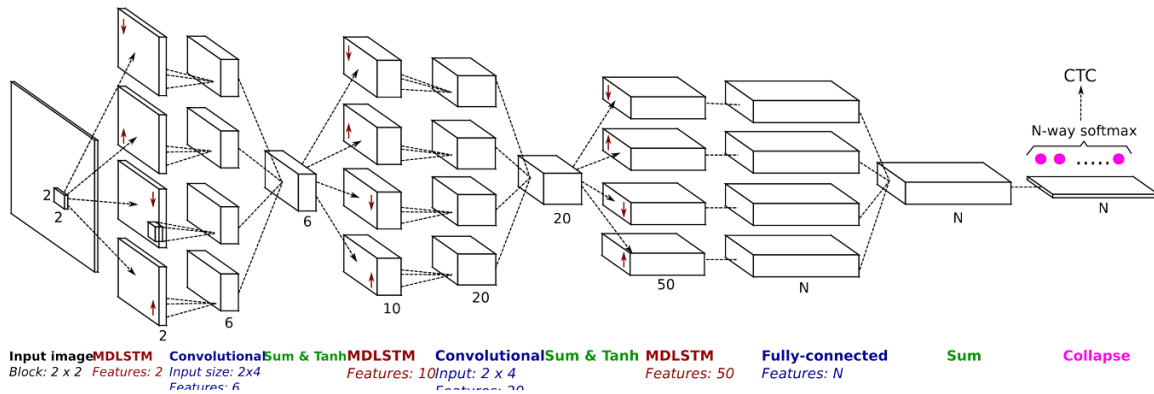
Cette méthode prend en compte le hors ligne donc l'aspect bidimensionnel. Ce modèle est inspiré de la reconnaissance vocale, la génération de légendes d'image et la traduction automatique. Le grand changement est dans l'application de ce mécanisme d'attention. Le mécanisme d'attention permet au modèle de se concentrer sur des parties de l'image à différents moments. Les auteurs du texte ont des résultats concluants avec des taux d'erreur faible, et il utilise les données IAM. IAM est une base de données souvent utilisée pour évaluer les systèmes de reconnaissance d'écriture manuscrite, cependant pour pouvoir utiliser les données il faut donner ses coordonnées (pour un suivi des données) que je n'ai pas réussi à avoir.



Ainsi notre programme se décompose en trois parties:

- Un encodeur qui transforme l'image 2D en cartes de caractéristiques
- Un mécanisme d'attention qui permet de focaliser dynamiquement sur différentes régions des cartes à chaque instant
- Un décodeur séquentiel qui fait la prédiction caractère par caractère en s'appuyant sur le mécanisme d'attention précédent

Encodeur MDLSTM



J'ai en entrée une image que je parcours dans les 4 directions, avec un réseau indépendant pour chaque directions. Comme le MDLSTM est un type de LSTM, j'ai en entrée 5 paramètre et en sortie que deux valeurs:

- un vecteur x (la position dans l'image)
- h correspondant à la sortie précédente
- q la direction de balayage

$$(h_{i,j}, q_{i,j}) = LSTM(x_{i,j}, h_{i,j\pm 1}, h_{i\pm 1,j}, q_{i,j\pm 1}, q_{i\pm 1,j})$$

Les couches de convolution permet de réduire la taille des cartes, comme pour le CNN, plus je monte dans le réseau plus les caractéristiques sont riches mais la résolution est plus petite.

Puis cela me donne une carte caractéristiques par étiquette qui est ensuite changé en une couche de "collapse" (d'écrasement) qui somme la verticale des cartes pour transformer les données 2D en 1D, puis je normalise cette séquence grâce au softmax (probabilité de chaque caractère à chaque position).

Puis je l'envoie dans un CTC (Connectionist Temporal Classification) qui ajoute un label "vide", supprime les répétitions et les blancs pour avoir une meilleure transcription.

Encodeur

Une première IA nommé encodeur qui est préalablement passé dans le MDLSTM que je propose se rapproche du CNN (Convolutional Neural Network) qui se décompose en 3 partie:

- *Couches de convolution* : Détection des caractéristiques.
- *Couches de pooling* : Réduction de la taille des données pour accélérer le traitement.
- *Couches entièrement connectées* : Prédiction finale.

Pour écrire un réseau où les données passent de façon linéaire à travers chaque couche, j'utiliserais dans [torch.nn](https://pytorch.org/docs/stable/nn.html), nn.Sequential qui chaîne plusieurs couches les unes à la suite des autres dans l'ordre.

Couches de convolution

Le but de cette partie est d'extraire des caractéristiques locales comme les bords, textures, motifs...

Pour cela j'utilise le filtre de Sobel, il est utilisé pour détecter les contours plus précisément il exagère les contours.

Pour cela on utilise le gradient d'intensité pour chaque pixel, ainsi le noyaux donne:

→ Pour la dérivée horizontale:

-1	0	1
-2	0	2
-1	0	1

→ Pour la dérivée verticale:

-1	-2	-1
0	0	0
1	2	1

Fonction: [Filtre Sobel](#)

Dans [torch.nn](#) nous avons la fonction `nn.Conv2d`(canal, filtres, taille du noyaux, réduction de l'image).

Cependant le désavantage que nous pouvons avoir est la non linéarité que je retire grâce au `nn.ReLU()` qui remplace tous les nombre négatif par 0 (le problème que je peux avoir est si il y a trop de nombre négatif le relu renvoie que des 0 et donc arrête d'apprendre).

Couches de pooling

Le but de cette partie est de réduire la taille spatiale tout en gardant les informations importantes dans l'image, ce qui permettra de réduire les calculs.

Pour cela j'utilise un filtre de pooling, il prend un bloc et parmi le bloc il garde la valeur la plus grande est remplace le bloc entier par cette valeur.

Fonction: [Filtre pooling](#)

Dans [torch.nn](#) nous avons la fonction `nn.MaxPool2d`(taille du noyaux, pas de déplacement).

Couche entièrement connecté

Dans le cas particulier de la reconnaissance de caractère, cette partie n'y est pas (et je ne l'ai pas dans mon code final). Cependant, il peut quand même être ajouté pour mieux apprendre les combinaisons:

Après la couche de pooling, il faut aplatir spatialement l'image c'est-à-dire passer de 2D en 1D, car le RNN ne prend pas des images mais des séquences de vecteurs. Ainsi on change $H \times W$ (taille spatiale de l'image) en T (en temporelles donc $H \times W = T$) avec des vecteur de taille C . Puis on réduit C par la dimension de `enc_out_dim`, `enc_out_dim` est une petite valeur permettant de réduire la charge de mémoire et uniformise la dimension à travers les modules.

Décodeur

Une seconde IA nommé encodeur que je propose se rapproche du LSTM (Long Short-Term Memory) qui se décompose en 3 partie:

- *Forget Gate* : Détermine si l'état précédent doit être conservé ou oublié.
- *Input Gate* : Intègre de nouvelles informations.
- *Output Gate* : Génère la sortie actuelle du réseau.

Cependant j'utilise un GRU(Gated Recurrent Unit), qui se rapproche du LSTM mais qui n'a pas de partie forget gate, input gate, et output gate explicite mais le fonctionnement est presque le même. Pour cela je prend dans [torch.nn](https://pytorch.org/docs/stable/nn.html#torch.nn.GRU) le `nn.GRU`(dimension de l'image et du mot encodé, dimension du mot encodé, batch_first), a l'intérieure on peut séparer le GRU en 3 parties:

- Premier calcul: Décide quelle partie du mot encodé précédent on garde
 $z_t = \sigma(W_z \cdot [x_t, h_{t-1}])$
- Deuxième calcul: Décide quelle partie du mot encodé on oublie pour créer un candidat
 $\tilde{h}_t = \tanh(W_h \cdot [x_t, \sigma(W_r \cdot [x_t, h_{t-1}]) * h_{t-1}])$
- Troisième calcul: Est l'interpolation entre le mot encodé précédent et le candidat
 $h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$

Or le décodeur est un type de RNN, il doit avoir une partie attention qui est vue après.

Attention

Cette partie est la deuxième modifiée par rapport à un système de reconnaissance traditionnelle. Donc le but de l'attention en générale est de calculer le score brut sur une position donnée. Elle à 3 paramètre d'entrée (1 sortie):

- la sortie de l'encodeur MDLSTM
- la sortie de l'attention précédent
- l'état précédent

Il faut normalisé en softmax avant de le donner au prochain instant:

$$\alpha_{(i,j),t} = \frac{e^{z_{(i,j),t}}}{\sum_{i',j'} e^{z_{(i',j'),t}}}$$

Puis une je crée une synthèse pondérée de l'image tel que:

$$g_t = \sum_{i,j} \alpha_{(i,j),t} e_{i,j}$$

Et cette sortie est utilisée en tant que donnée dans le LSTM.

(Reconnaissance final)

La reconnaissance final que je proposerais se rapproche du CTC (Connectionist Temporal Classification) qui utilise le *Vanilla Beam Search* :

- *WorldBeamSearch*: Sélectionne le mot le plus proche

Source

- [1] Travaux de Melchior OBAME OBAME et Saad BENDAOUD sur les réseaux de neurones pour la reconnaissance de l'écriture manuscrite :
<https://github.com/saadbenda/Neural-network-for-handwriting>
- [2] Référence du traitement d'image:
Annexe 1: [Digital Image Processing](#)
- [3] Liste exhaustive des formats d'image:
https://en.wikipedia.org/wiki/Image_file_format
- [4] Conversion de RGB/XYZ:
http://brucelindbloom.com/index.html?Eqn_RGB_XYZ_Matrix.html
- [5] Liste exhaustive des méthodes de seuillage dynamique:
[https://en.wikipedia.org/wiki/Thresholding_\(image_processing\)](https://en.wikipedia.org/wiki/Thresholding_(image_processing))
- [6] Extension de Niblack et Sauvola:
Annexe 2: [Adaptive_Document_Binarization](#)
- [7] Travaux de Théodore Bluche, Jérôme Louradour et Ronaldo Messina sur "Scan, Attend and Read: End-to-End Handwritten Paragraph Recognition with MDLSTM Attention":
<https://arxiv.org/pdf/1604.03286>